Vite & Gourmand

Documentation Technique

TP Développeur Web et Web Mobile

Candidat : NAZE ABDULLA Sorinen

1. Réflexions technologiques initiales

Avant de démarrer le développement, j'ai analysé les besoins du cahier des charges afin de choisir la stack technique la plus adaptée au projet.

Choix du frontend

J'ai choisi **Vue.js 3** avec la Composition API plutôt que React ou Angular pour plusieurs raisons : sa courbe d'apprentissage progressive, sa documentation claire en français, et sa légèreté pour une application de taille moyenne. Le système de réactivité intégré (ref, computed, watch) permet de gérer les filtres dynamiques sans rechargement de page, exigence explicite du cahier des charges.

Choix du backend

FastAPI a été retenu pour sa performance (basé sur Starlette/ASGI), sa génération automatique de documentation OpenAPI accessible à /docs, et sa validation automatique des données via Pydantic. SQLAlchemy assure l'ORM pour PostgreSQL, permettant des requêtes typées et maintenables.

Double base de données

Le cahier des charges impose l'usage d'une base relationnelle ET d'une base non relationnelle. **PostgreSQL** stocke toutes les données métier (utilisateurs, commandes, menus, avis). **MongoDB** stocke les statistiques de commandes par menu pour alimenter les graphiques du dashboard admin — cas d'usage idéal pour un document store NoSQL (agrégations rapides, pas de jointures complexes).

Déploiement

Le VPS Hostinger et le domaine **zotpresta.re** existaient avant ce projet (réutilisés d'un projet personnel antérieur). **Dokploy** a été choisi comme PaaS auto-hébergé pour simplifier le déploiement via Docker Compose, avec **Traefik** comme reverse proxy gérant automatiquement les certificats SSL.

2. Configuration de l'environnement de travail

Environnement local

Outil	Version	Usage
Node.js	20.x LTS	Runtime frontend
Python	3.11	Runtime backend
PostgreSQL	15	Base de données relationnelle
MongoDB	6.0	Base de données NoSQL
Docker Desktop	4.x	Conteneurisation locale
VS Code	latest	Éditeur de code
Git	2.x	Gestion de versions

Démarrage en local

Backend :

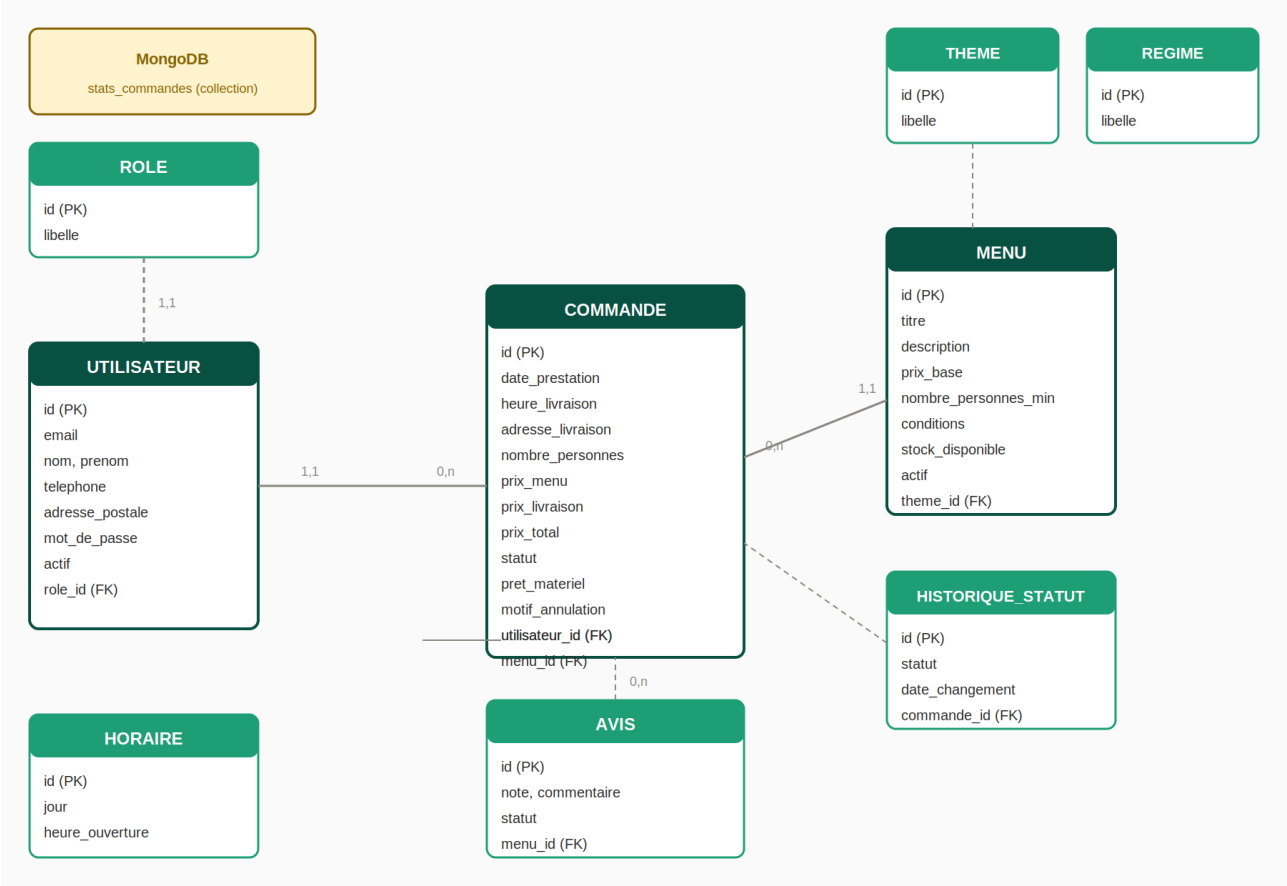
```
cd backend && python -m venv venv && source venv/bin/activate
pip install -r requirements.txt
uvicorn app.main:app --reload
```

Frontend :

```
cd frontend && npm install && npm run dev
```

3. Modèle Conceptuel de Données (MCD)

Le schéma ci-dessous représente les entités principales et leurs relations dans la base PostgreSQL.



MCD — Base de données relationnelle PostgreSQL

4. Diagramme de cas d'utilisation

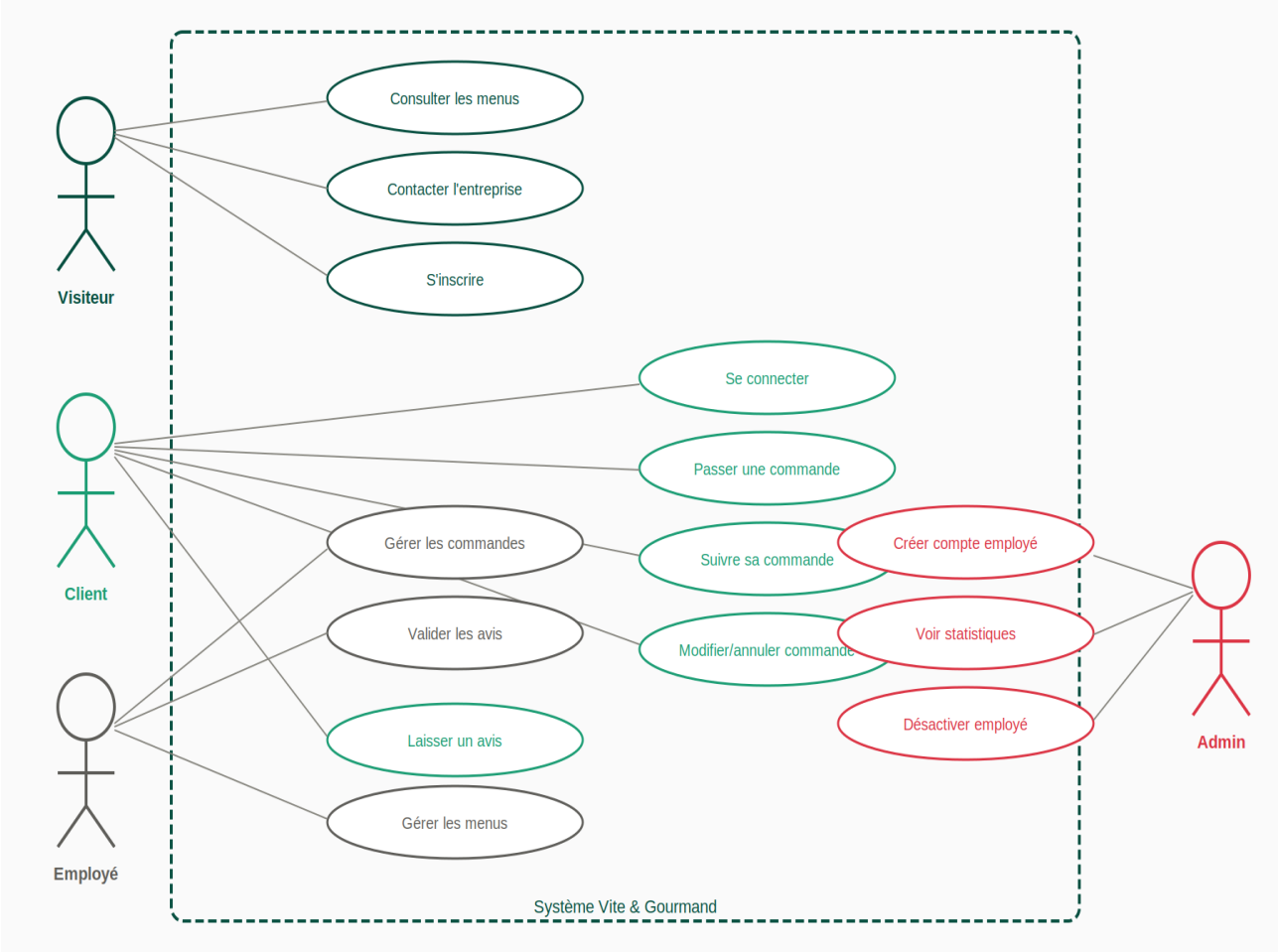


Diagramme de cas d'utilisation

5. Diagramme de séquence — Passage de commande

Ce diagramme illustre les interactions entre le client, le frontend Vue.js, l'API FastAPI et la base de données lors du passage d'une commande.

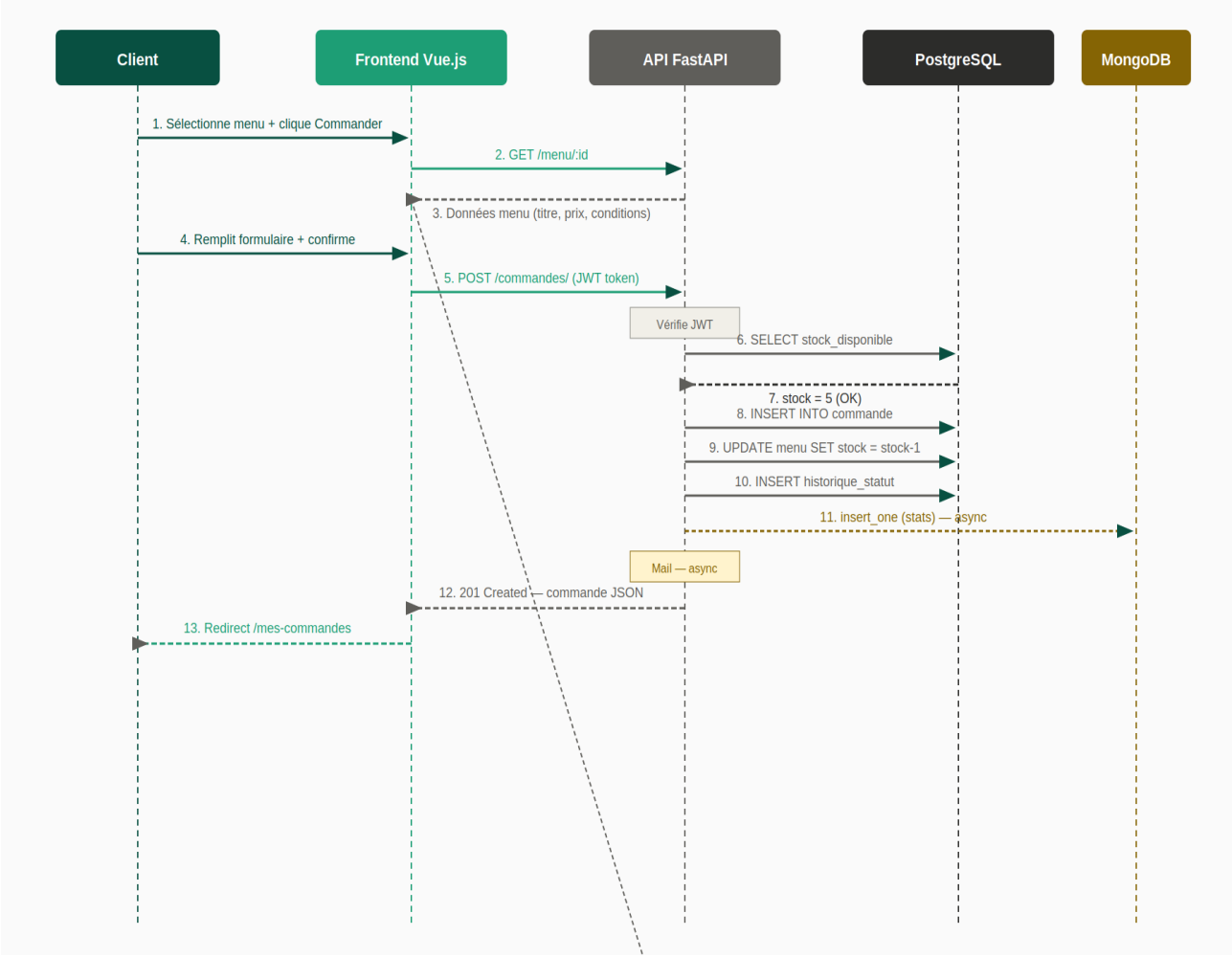


Diagramme de séquence — Passage d'une commande

6. Documentation du déploiement

Architecture de déploiement

L'application est déployée sur un VPS Hostinger via Dokploy. Dokploy agit comme un PaaS auto-hébergé (similaire à Heroku) qui orchestre les conteneurs Docker et gère les déploiements via GitHub. Traefik sert de reverse proxy et gère automatiquement les certificats SSL via Let's Encrypt.

Composant	Technologie	URL / Port
Frontend	Vue.js + Nginx (Docker)	https://zotpresta.re
Backend API	FastAPI + Uvicorn (Docker)	https://api.zotpresta.re
Base PostgreSQL	PostgreSQL 15 (Docker)	Port 5432 (interne)
Base MongoDB	MongoDB 6 (Docker)	Port 27017 (interne)
Reverse proxy	Traefik	Port 80/443
Déploiement	Dokploy	Interface admin VPS

Processus de déploiement

1. Push du code sur la branche **sori** de GitHub
2. Dokploy détecte le push via webhook GitHub
3. Dokploy reconstruit les images Docker (frontend + backend)
4. Dokploy relance les conteneurs avec Docker Compose
5. Traefik route automatiquement le trafic vers les bons conteneurs
6. Les certificats SSL sont renouvelés automatiquement par Let's Encrypt

Note sur le domaine

Le domaine **zotpresta.re** et le VPS Hostinger existaient avant ce projet ECF — ils ont été réutilisés d'un projet personnel antérieur pour éviter des coûts supplémentaires. Ce choix est documenté dans le dossier ECF et n'affecte pas le fonctionnement de l'application.

Variables d'environnement (production)

Variable	Description
`DATABASE_URL`	`postgresql://postgres:postgres@postgres:5432/vite_gourmand`
`MONGO_URL`	`mongodb://mongo:27017`
`SECRET_KEY`	`Clé JWT de production (256 bits)`
`MAIL_SERVER`	`smtp.hostinger.com (port 465)`
`MAIL_USERNAME`	`contact@zotpresta.re`
`FRONTEND_URL`	`https://zotpresta.re`